

Docs Chain — Configuration Schema Reference

Revision: 2 **Last modified:** 2026-05-29T12:00:00Z **Status:** Design documentation — the schema is PLANNED (Phase 4 — config loader + validation). Describes the DESIGNED YAML contract. **Authority:** Operator mandate 2026-05-29 (Docs Chain initiative) **Design provenance:** authoritative Phase-0 DESIGN / RESEARCH / PLAN live in the consuming project research tree (docs/research/docs_chain/); this document is the self-contained specification.

Status

PLANNED (Phase 4). The YAML loader and validator land in Phase 4 (internal/config). This document is the formal contract a consuming project writes against; it is not a claim that the loader parses it today.

1. File location and discovery

- One YAML file per context under `.docs_chain/contexts/<name>.yaml` at the project root.
 - The filename stem SHOULD match the `context:` field value.
 - Each context is independent: `docs_chain sync <name>` loads only `<name>.yaml`; `docs_chain sync --all` loads every file in the directory.
-

2. Top-level document schema

<code>context:</code>	<code><string></code>	<i># REQUIRED — unique context name</i>
<code>description:</code>	<code><string></code>	<i># OPTIONAL — human-readable purpose</i>
<code>nodes:</code>	<code><map></code>	<i># REQUIRED — node-id -> node-spec</i>
<code>edges:</code>	<code><list></code>	<i># REQUIRED — list of edge-spec</i>
<code>transforms:</code>	<code><map></code>	<i># REQUIRED if any edge names a</i>
	<i>transform</i>	

Field	Type	Required	Default	Validation
context	string	yes	—	non-empty; SHOULD match filename stem; unique across .docs_chain/ contexts/
description	string	no	""	free text ≥1 entry;
nodes	map<string, node-spec>	yes	—	keys are node ids, unique within the context
edges	list	yes	—	derive-from sub-graph MUST be acyclic
transforms	map<string, transform- spec>	conditional	{}	every transform name referenced by an edge MUST be defined here

3. Node spec

```
<node_id>: { kind: <node-kind>, path: <relative-path> }
```

Field	Type	Required	Default	Validation
kind	enum	yes	—	one of the node kinds below
path	string	yes	—	project-root- relative path to the artefact REQUIRED only for kind:
members	string (glob)	conditional	—	fingerprint; glob enumerating the roster/corpus members (§11.4.86)

Field	Type	Required	Default	Validation
exclude	list	no	[]	for kind: fingerprint: member globs to exclude (e.g. gitignored downloaded/)

3.1 Allowed kind values

Value	Role	Direction
markdown	canonical .md source	input
html	pandoc export	derived
pdf	weasyprint export	derived
sqlite	database	input + derived (bidirectional)
summary	generated Markdown digest	derived
status	§11.4.45 status doc	derived
status_summary	§11.4.56 two- audience digest	derived
fingerprint	roster/corpus member-list hash sidecar (§11.4.86)	input

4. Edge spec

Two edge shapes. The type field selects which.

4.1 derive-from (one-way)

```
- { type: derive-from, from: <node_id>, to: <node_id>, transform:
  <transform_name> }
```

Field	Type	Required	Default	Validation
type	literal derive- from	yes	—	—

Field	Type	Required	Default	Validation
from	node id	yes	—	MUST exist in nodes; may be a list (multi-input transform)
to	node id	yes	—	MUST exist in nodes; MUST NOT introduce a cycle
transform	transform name	yes	—	MUST exist in transforms

4.2 sync (bidirectional)

```
- { type: sync, a: <node_id>, b: <node_id>, authority: <node_id>,
  transform_a_to_b: <transform_name>, transform_b_to_a:
    <transform_name> }
```

Field	Type	Required	Default	Validation
type	literal sync	yes	—	—
a	node id	yes	—	MUST exist in nodes
b	node id	yes	—	MUST exist in nodes; $a \neq b$
authority	node id	yes	—	MUST equal a or b; the default source of truth (ARCHITECTURE §5)
transform_a_to_b	transform name	conditional	derived from kinds when unambiguous	regenerates b from a
transform_b_to_a	transform name	conditional	derived from kinds when unambiguous	regenerates a from b

When the two transforms are omitted and the (kind_a, kind_b) pair maps to a known builtin pair (e.g. markdown ↔ sqlite), Docs Chain selects the builtin pair automatically; otherwise both MUST be specified.

5. Transform spec

A transform is either a builtin or an external command.

```
<transform_name>: { builtin: <builtin-name> }
```

```
# OR
```

```
<transform_name>: { exec: <command>, args: [<arg>, ...] }
```

Field	Type	Required	Default	Validation
builtin	enum	one-of	—	mutually exclusive with exec
exec	string	one-of	—	a command/script path, project-root-relative or on PATH; mutually exclusive with builtin
args	list	no	[]	only with exec; appended after the input/output paths Docs Chain passes

Exactly one of builtin / exec MUST be present.

5.1 Allowed builtin values

Builtin	Maps	Tool
pandoc-html	markdown → html	pandoc
weasyprint-pdf	html → pdf or markdown → pdf	weasyprint
colorize-html	html → html post-process (\$11.4.23)	internal
gen-summary	markdown → summary (pluggable generator)	configured generator
md-to-sqlite	markdown → sqlite	configured DB binary
sqlite-to-md	sqlite → markdown	configured DB binary
members-fingerprint		

Builtin	Maps	Tool
	members glob → fingerprint (\$11.4.86)	internal sha256-of-sorted-members

5.2 exec: transform contract

An exec: transform is invoked with input path(s) and the staged output temp path supplied by Docs Chain. It MUST:

- read ONLY from the declared input node paths;
- write its result to the staged temp path Docs Chain provides (never directly to the live artefact — Docs Chain owns the atomic rename, §8);
- exit 0 on success, non-zero on failure (triggers rollback, exit 3);
- produce **byte-stable** output for identical input (§11.4.50 — a non-deterministic transform causes false drift and fails the Phase 2 meta-test).

6. Annotated example — derive-from chain

The universal markdown-export chain (§11.4.65): one source, two exports.

```
context: guide                                     # context name (=
          filename stem)
description: Markdown doc with html + pdf
nodes:
  guide_md:  { kind: markdown, path: docs/guides/MY_GUIDE.md }
             # input
  guide_html: { kind: html,      path: docs/guides/MY_GUIDE.html }
             # derived
  guide_pdf:  { kind: pdf,       path: docs/guides/MY_GUIDE.pdf }
             # derived
edges:
  # one-way: regenerate html from md
  - { type: derive-from, from: guide_md, to: guide_html,
      transform: md-to-html }
  # one-way: regenerate pdf from html (so styling/colorization
    carries through)
  - { type: derive-from, from: guide_html, to: guide_pdf,
      transform: html-to-pdf }
transforms:
  md-to-html: { builtin: pandoc-html }
  html-to-pdf: { builtin: weasyprint-pdf }
```

Propagation: edit guide_md → guide_html regenerates → guide_pdf regenerates. Edit nothing → early-cutoff → exit 0, no writes.

7. Annotated example — sync edge

The issues chain (§11.4.93 / §11.4.12): DB is the single source of truth, bidirectionally synced with Markdown, then exported.

```
context: issues
description: Workable-items tracker chain
nodes:
  items_db:      { kind: sqlite,  path: docs/
                  workable_items.db } # input + derived
  issues_md:     { kind: markdown, path: docs/
                  Issues.md }         # input + derived
  issues_summary: { kind: summary, path: docs/
                  Issues_Summary.md } # derived
  issues_html:   { kind: html,    path: docs/
                  Issues.html }      # derived
  issues_pdf:    { kind: pdf,     path: docs/
                  Issues.pdf }       # derived
edges:
  # bidirectional single-source-of-truth; DB wins when it is the
  # sole dirty side
  - { type: sync, a: issues_md, b: items_db, authority: items_db,
      transform_a_to_b: md-to-db, transform_b_to_a: db-to-md }
  # one-way derivations from the resolved markdown view
  - { type: derive-from, from: issues_md, to: issues_summary,
      transform: gen-issues-summary }
  - { type: derive-from, from: issues_md, to: issues_html,
      transform: md-to-html }
  - { type: derive-from, from: issues_html, to: issues_pdf,
      transform: html-to-pdf }
transforms:
  md-to-db:      { exec: "scripts/testing/workable_items",
                  args: ["sync", "md-to-db"] }
  db-to-md:      { exec: "scripts/testing/workable_items",
                  args: ["sync", "db-to-md"] }
  gen-issues-summary: { exec: "scripts/testing/
                          generate_issues_summary.sh" }
  md-to-html:     { builtin: pandoc-html }
  html-to-pdf:    { builtin: weasyprint-pdf }
```

Conflict rule: if BOTH issues_md and items_db are dirty in one run, Docs Chain emits a conflict (exit 2, no writes) per ARCHITECTURE §5.

8. Validation rules summary

The loader (Phase 4) rejects a context (exit 4) when any holds:

1. context is empty or missing.
2. A node id is referenced by an edge but absent from nodes.
3. A transform name is referenced by an edge but absent from transforms.
4. A transform spec has neither builtin nor exec (or has both).
5. A fingerprint node lacks members.
6. A sync edge's authority is neither a nor b.
7. The derive-from sub-graph contains a cycle (Kahn residual).
8. Two nodes share the same id.